SOUQ

تقرير التحليل الشامل للمشروع وتحديد المشاكل وخطة الحل

تحليل أمني وتقني معمق لنظام التجارة الإلكترونية

Django REST Framework + React + TypeScript

تاريخ التقرير: 23 أبريل 2026

إجمالي المشاكل المكتشفة: 93 مشكلة

التصنيفات: حرجة / عالية / متوسطة / منخفضة / معمارية

فهرس المحتويات

1. نظرة عامة على المشروع

1.1 البنية التقنية

1.2 إحصائيات المشاكل

2. المشاكل الحرجة (Critical) - 12 مشكلة

2.1 مشاكل حرجة في الواجهة الخلفية (Backend)

2.2 مشاكل حرجة في الواجهة الأمامية (Frontend)

3. المشاكل العالية الخطورة (High) - 18 مشكلة

3.1 مشاكل عالية في الواجهة الخلفية

3.2 مشاكل عالية في الواجهة الأمامية

4. المشاكل المتوسطة (Medium) - 31 مشكلة

4.1 ملخص مشاكل الواجهة الخلفية

4.2 ملخص مشاكل الواجهة الأمامية

5. المشاكل المنخفضة (Low) - 27 مشكلة

6. المخاوف المعمارية (5 قضايا)

7. خارطة طريق إصلاح الأولويات

1. نظرة عامة على المشروع

1.1 البنية التقنية

الواجهة الخلفية (Backend)

Django 5.2 + Django REST Framework 3.17 مع PostgreSQL كقاعدة بيانات. يستخدم SimpleJWT للمصادقة عبر ملفات تعريف الارتباط HttpOnly. يحتوي على 7 تطبيقات Django: الحسابات، الكتالوج، السلة، الطلبات، المفضلة، التقييمات، والأدوات المشتركة. يدعم مصادقة OAuth عبر Google وFacebook، ونظام OTP للتحقق من عنوان IP الجديد، ونظام إدارة للمشرفين مع سجل تدقيق كامل.

الواجهة الأمامية (Frontend)

React 18 + TypeScript + Vite 5 مع Tailwind CSS. يستخدم Zustand لإدارة الحالة وAxios مع اعتراض لتحديث الرمز التمييزي تلقائيًا. يحتوي على 32 صفحة و28 مكونًا. يدعم واجهة عربية من اليمين لليسار بالكامل مع نظام داكن/فاتح. يتضمن لوحة تحكم للتاجر والمشرف مع تحميل كسول.

نظام المصادقة

نظام JWT مقسم: رمز الوصول في الذاكرة فقط (آمن)، رمز التحديث في ملف تعريف ارتباط HttpOnly. دعم تسجيل الدخول عبر البريد الإلكتروني/كلمة المرور، Google OAuth، Facebook OAuth. نظام تحقق OTP عبر البريد الإلكتروني عند تسجيل الدخول من عنوان IP جديد. نظام استئناف للمستخدمين المجمدين مع فترة 14 يومًا.

ميزات الأعمال

سوق إلكتروني جزائري متكامل: كتالوج منتجات مع متغيرات وأصناف، سلة تسوق، نظام طلبات مع تتبع، طرق دفع متعددة (COD، بطاقة، BaridiMob، CCP)، نظام تقييمات للمنتجات والبائعين والمشترين، لوحة تحكم للتاجر والمشرف، نظام بلاغات واستئنافات.

1.2 إحصائيات المشاكل

12	18	31	27+5
حرجة	عالية	متوسطة	منخفضة + معمارية

تم اكتشاف إجمالي 93 مشكلة عبر تحليل شامل لكل من الواجهة الخلفية والواجهة الأمامية. تشمل المشاكل الحرجة ثغرات أمنية خطيرة ومشاكل تسلسل بيانات قد تؤدي إلى خسائر مالية. يُنصح بالبدء بإصلاح المشاكل الحرجة فورًا قبل متابعة أي تطوير آخر.

الفئة	Backend	Frontend	الإجمالي
حرجة (Critical)	5	7	**12**
عالية (High)	8	10	**18**

الفئة	Backend	Frontend	الإجمالي
متوسطة (Medium)	16	15	**31**
منخفضة (Low)	12	15	**27**
معمارية (Arch)	5	-	**5**

2. المشاكل الحرجة (Critical)

2.1 الواجهة الخلفية - 5 مشاكل حرجة

accounts/views.py (324-428)

C1-BE

تسجيل الدخول الاجتماعي بدون التحقق من الرمز - استحواذ على الحسابات

نقطة نهاية social_login تقبل البريد الإلكتروني ومزود الخدمة ومعرف المزود مباشرة من طلب العميل دون التحقق من أي رمز OAuth مقابل خوادم Google/Facebook. يمكن لمهاجم إرسال طلب POST يحتوي على بريد إلكتروني الضحية والحصول على وصول كامل إلى حسابه أو إنشاء حساب جديد يمنع المالك الحقيقي من الدخول.

التأثير: استحواذ كامل على أي حساب يُعرف بريده الإلكتروني. ثغرة أمنية حرجة تسمح بانتحال الهوية.

خطة الحل:

التحقق من رمز الوصول الخاص بـ Google أو Facebook من جانب الخادم باستخدام نقاط نهاية التحقق من الرمز المميز لكل مزود (tokeninfo لـ Google، /me?access_token لـ Facebook) قبل قبول أي مطالبة هوية. رفض أي طلب لا يحتوي على رمز OAuth صالح.

orders/views.py (70-111)

C2-BE

حالة سباق في إنشاء الطلبات - تجاوز المخزون السلبي

داخل transaction.atomic()، يتم فحص المخزون (stock < quantity) ثم خصمه لاحقًا دون استخدام قفل على مستوى الصف. يمكن لطلبين متزامنين اجتياز فحص المخزون كلاهما وخصم الكمية، مما يؤدي إلى مخزون سلبي وتجاوز البيع الفعلي للمنتجات المتاحة.

التأثير: بيع منتجات غير متوفرة فعليًا، فساد البيانات، خسائر مالية مباشرة للبائعين.

خطة الحل:

استخدام select_for_update() على كائنات ProductVariant قبل فحص وخصم المخزون، أو استخدام تعبيرات F() مع شرط فلاتر يضمن stock >= quantity قبل التحديث.

cart/views.py (37-58)

C3-BE

حالة سباق في عمليات السلة - تجاوز المخزون

دالة cart_add تتحقق من المخزون وتنشئ/تحديث عناصر السلة دون استخدام select_for_update(). يمكن لطلبات متزامنة متعددة تجاوز المخزون المتاح في السلة، مما يؤدي إلى فشل الدفع لاحقًا أو تجاوز البيع أثناء إنشاء الطلب.

التأثير: تجربة مستخدم سيئة (فشل الدفع) أو تجاوز بيع عند إنشاء الطلب.

خطة الحل:

قفل صف المتغير باستخدام select_for_update() قبل فحص المخزون، أو استخدام تعبيرات F() الذرية مع شرط الفلترة.

accounts/views.py (259-291)

C4-BE

المستخدمون المجمودون يمكنهم تجديد الرموز بشكل لا نهائي

CustomTokenRefreshView يقرأ رمز التحديث من ملفات تعريف الارتباط ويصدر رموز وصول جديدة دون التحقق من حقل status للمستخدم. مستخدم مجمّد لديه رمز تحديث صالح يمكنه الحصول على رموز وصول طازجة باستمرار، مما يجعل كل عمليات التجميد عديمة الفائدة.

التأثير: تجاوز كامل لنظام تجميد الحسابات. فئة المصادقة ووسيطة التجميد لا فائدة منها.

خطة الحل:

بعد التحقق من رمز التحديث، استرجاع المستخدم والتحقق من status. إذا كان مجمّدًا، إضافة الرمز إلى القائمة السوداء وإرجاع 403.

orders/views.py (169-180)

C5-BE

إلغاء الطلب لا يستعيد المخزون

عند إلغاء طلب (order_cancel)، يتم تحديث حالة الطلب فقط دون استعادة المخزون الذي تم خصمه عند الإنشاء. المخزون المخصوم لا يُعاد أبدًا، مما يؤدي إلى خسارة دائمة في الجرد.

التأثير: خسارة جرد دائمة مع كل إلغاء طلب. بيانات المخزون تصبح غير دقيقة تدريجيًا.

خطة الحل:

داخل معاملة ذرية، التكرار على عناصر الطلب وزيادة مخزون المتغيرات بكمية الإلغاء. معالجة حالات الطلب الجزئي إذا لزم الأمر.

2.2 الواجهة الأمامية - 7 مشاكل حرجة

C1-FE

authStore.ts (221-228)

isAuthenticated محفوظ في localStorage - اختطاف الجلسة

يتم حفظ isAuthenticated في localStorage تحت مفتاح auth-storage. عند إعادة تحميل الصفحة، يعتقد التطبيق أن المستخدم مسجل الدخول حتى عندما يكون رمز الوصول فارغًا (لأنه في الذاكرة فقط). هذا يؤدي إلى وميض المحتوى المصادق عليه ومكالمات API فاشلة صامتة (401).

التأثير: تجربة مستخدم سيئة ومكالمات API غير ضرورية. يمكن استغلاله لعرض محتوى محمي مؤقتًا.

خطة الحل:

عدم حفظ isAuthenticated في التخزين المحلي. الاشتقاق من accessToken !== null فقط. حفظ user/profile فقط كبيانات مخزنة مؤقتة.

C2-FE

authStore.ts (111-115)

رموز معلقة في sessionStorage - سرقة الرموز

يتم تخزين رموز الوصول والتحديث الكاملة في sessionStorage أثناء تدفق إكمال الملف الشخصي. أي هجوم XSS على أي صفحة يمكنه استخراج هذه الرموز. تخزين رمز التحديث في تخزين يمكن الوصول إليه عبر JavaScript يهدم الغرض من ملف تعريف الارتباط HttpOnly.

التأثير: إذا حدث هجوم XSS، يمكن للمهاجم الحصول على رمز التحديث الكامل واختراق الحساب بشكل دائم.

خطة الحل:

يجب أن يقوم الواجهة الخلفية بتعيين ملف تعريف ارتباط رمز التحديث قبل إعادة التوجيه إلى إكمال الملف الشخصي. إذا تم إرجاعه في نص الاستجابة، فلا تخزن سوى رمز الوصول في الذاكرة.

C3-FE

authService.ts (70-75)

حالة سباق في تحديث الرمز قد تفقد الطلبات

في معترض Axios، بعد نجاح تحديث الرمز، يتم تعيين isRefreshing = false قبل اكتمال إعادة محاولة الطلب الأصلي. إذا ورد استجابة 401 أخرى بين إعادة التعيين والاكتمال، سيتم بدء تحديث ثانٍ مما قد يسبب تعارضات أو فقدان طلبات.

التأثير: فقدان طلبات المستخدم أو سلوك غير متوقع أثناء التحديث المتزامن للرموز.

خطة الحل:

نقل isRefreshing = false إلى كتلة finally، أو عدم إعادة التعيين حتى تكتمل إعادة المحاولة بالكامل.

authService.ts (83)

C4-FE

التنقل عبر window.location.href يمحو كل الحالة

عند فشل تحديث الرمز، يتم استخدام 'window.location.href = '/login وهو إعادة تحميل كاملة للصفحة تدمر حالة React بالكامل، وتلغي الطلبات المعلقة، وتسبب تسربًا محتملًا للذاكرة من الوعود التي لم تُلغ.

التأثير: تجربة مستخدم سيئة مع تحميل كامل غير ضروري وفقدان حالة التطبيق.

خطة الحل:

استخدام تنقل React Router بدلاً من إعادة التحميل الكاملة. استيراد وظيفة التنقل من الموجه.

App.tsx (126, 159)

C5-FE

مسار مكرر لـ /registration-success - عام ومحمي

يوجد مساران لنفس المسار: واحد عام (خط 126) وآخر محمي عبر PrivateRoute (خط 159). في React Router v6، المسار العام يفوز، مما يجعل الصفحة متاحة بدون مصادقة. أي منطق حماية داخل المكون يتم تجاوزه تمامًا.

التأثير: تجاوز حماية المصادقة على مسار يفترض أن يكون محميًا.

خطة الحل:

إزالة المسار العام والاحتفاظ بالمسار المحمي فقط، أو نقل المسار العام داخل كتلة التخطيط المناسب حسب القصد.

Cart.tsx (135), Checkout.tsx (69)

C6-FE

حساب الشحن من جانب العميل - عدم تطابق مع الواجهة الخلفية

تكلفة الشحن محسوبة ثابتة على الواجهة الأمامية (500 دج إذا المجموع أقل من 5000 دج، مجانًا غير ذلك). إذا كانت الواجهة الخلفية لديها قواعد مختلفة (حسب الولاية، الوزن، البائع)، فإن المبلغ المعروض للمستخدم سيختلف عن ما يتم خصمه فعليًا.

التأثير: المستخدم يؤكد بمبلغ X لكن يُحمل بمبلغ Y. خلافات مالية ونقص ثقة.

خطة الحل:

جلب تكلفة الشحن من نقطة نهاية الواجهة الخلفية /orders/shipping-cost مع معاملات الولاية والوزن، أو عرض "الإجمالي التقديري" مع إخلاء المسؤولية.

OrderDetail.tsx (107), types/index.ts

C7-FE

نوع OrderItem يفتقر لحقول البائع - تعطل التجميع

نوع TypeScript OrderItem لا يحتوي على حقول seller_id أو seller_name. منطق التجميع في OrderDetail.tsx يستخدم item.seller_id وهو دائمًا undefined، مما يؤدي إلى تجميع كل العناصر تحت مفتاح undefined. تفاصيل البائع وإثباتات الدفع وبيانات CCP لا تعمل.

التأثير: صفحة تفاصيل الطلب معطلة تمامًا - لا يمكن رؤية تفاصيل البائعين.

خطة الحل:

إضافة seller_id و seller_name و ccp_number و ccp_name و baridimob_id إلى نوع OrderItem في types/index.ts والتأكد من أن المُسلسل في الواجهة الخلفية يعيدها.

3. المشاكل العالية الخطورة (High)

3.1 الواجهة الخلفية - 8 مشاكل عالية

accounts/views.py (434-477)

H1-BE

التحقق من OTP بدون تقييد المعدل - قابل للاختراق بالقوة الغاشمة

نقطة نهاية verify_ip_login تقبل بريدًا إلكترونيًا ورمز OTP من 6 أرقام بدون أي فئة تقييد. مساحة الرمز هي 900,000 تركيبة محتملة. مع محاولات غير محدودة، يمكن اختراق الرمز بالقوة الغاشمة.

التأثير: استحواذ على الحسابات عبر اختراق OTP بالقوة الغاشمة.

خطة الحل:

إضافة فئة تقييد مخصصة (5 محاولات/دقيقة) وتنفيذ عداد محاولات على OTPVerification مع قفل بعد N محاولة فاشلة.

orders/views.py (185-190)

H2-BE

نقطة تتبع الطلبات تكشف بيانات المشتري الحساسة

order_track ليس لديها فئة أذونات محددة. أي مستخدم مصادق عليه يمكنه معرفة رقم الطلب ورؤية الطلب الكامل بما في ذلك تفاصيل الشحن (الاسم، الهاتف، العنوان، الولاية). رقم الطلب بتنسيق قابل للتخمين (ORD + 8 أرقام).

التأثير: كشف بيانات شخصية لجميع العملاء. انتهاك خصوصية.

خطة الحل:

طلب أن يكون الطالب هو مالك الطلب أو مشرفًا. للتتبع العام، إرجاع الحالة والجدول الزمني فقط دون بيانات الشحن الشخصية.

accounts/views.py (173, 644, 755, 818)

H3-BE

تسريب رسائل الخطأ الداخلية للعملاء

نقاط نهاية متعددة تلتقط Exception وتُرجع str(e) مباشرة في نص الاستجابة. يمكن أن يتضمن ذلك تتبعات المكدس، سلاسل اتصال قاعدة البيانات، ومسارات الملفات.

التأثير: كشف معلومات داخلية للمهاجمين تساعد في تخطيط هجمات أكثر تقدمًا.

خطة الحل:

عدم إرجاع رسائل الاستثناء أبدًا. تسجيل الخطأ الكامل في الخادم وإرجاع رسالة خطأ عامة للعميل.

orders/views.py (304-323)

H4-BE

التاجر يمكنه تعيين أي حالة طلب - لا تحقق من آلة الحالة

merchant_order_status يقبل أي قيمة من حالة الطلب دون التحقق مما إذا كان الانتقال صالحًا. يمكن للتاجر إعادة تعيين حالة من "تم التسليم" إلى "قيد الانتظار" أو تعيين "مسترد" على أي طلب.

التأثير: فساد منطقي لحالة الطلب. يمكن للتاجر التلاعب بالحالات للإحتيال.

خطة الحل:

تنفيذ آلة حالة تسمح فقط بالانتقالات الصالحة (قيد الانتظار ← مؤكد ← قيد المعالجة ← تم الشحن ← تم التسليم، مع إلغاء/استرداد من حالات محددة).

orders/views.py (277-284)

H5-BE

رفض إثبات الدفع يتجاوز حالة الدفع للطلب بالكامل

عندما يرفض بائع واحد إثبات الدفع، يتم تعيين حالة الدفع للطلب بالكامل إلى "مرفوض". في طلب متعدد البائعين، قد يتجاوز هذا حالة "تم الرفع" أو حتى "مدفوع" من بائع آخر.

التأثير: حالة دفع الطلب غير صحيحة، ارتباك مالي محتمل.

خطة الحل:

تحديث حالة الدفع إلى "مرفوض" فقط إذا تم رفض جميع الإثباتات. خلاف ذلك، الاحتفاظ بـ "تم الرفع".

accounts/views.py (21-32)

H6-BE

رفع ملفات Base64 بدون تحقق - رفع ملفات تعسفي

دالة decode_base64_file تفصل على ;base64, وتنشئ ملفًا بالامتداد المستخرج من URI البيانات. لا يوجد تحقق من نوع الملف أو حجمه أو MIME type. يمكن رفع ملفات خطرة مقنعة بصور.

التأثير: تنفيذ رمز عن بُعد محتمل إذا كانت ملفات الوسائط تُقدم من خادم ضعيف. هجوم حجب الخدمة عبر ملفات ضخمة.

خطة الحل:

التحقق من MIME type مقابل قائمة مسموحات (JPEG, PNG, WEBP)، فرض حجم أقصى، واستخدام Pillow أو python-magic للتحقق من أن الملف صورة حقيقية.

accounts/serializers.py (39-48, 249-254)

H7-BE

تعريف مكرر لـ AdminActionLogSerializer - كشف حقول غير مقصودة

يوجد تعريفان لنفس اسم الفئة. الثاني يستخدم __all__ = fields__' الذي قد يكشف حقول غير مقصودة للواجهة البرمجية مثل حالات JSON الخام before_state/after_state أو معرفات داخلية.

التأثير: سلوك API غير متسق حسب ترتيب الاستيراد. تسريب بيانات محتمل.

خطة الحل:

إزالة التعريف المكرر. الاحتفاظ بتعريف واحد مع قائمة حقول صريحة.

accounts/views.py (186-197)

H8-BE

تأكيد إعادة تعيين كلمة المرور يتجاوز مدققات كلمة المرور

password_reset_confirm يقبل new_password من نص الطلب وينفذ user.set_password(new_password) مباشرةً دون استدعاء validate_password(). يمكن للمستخدم تعيين كلمات مرور ضعيفة جدًا عبر إعادة التعيين.

التأثير: مستخدمون بكلمات مرور ضعيفة رغم إعدادات المدقق في المشروع.

خطة الحل:

استيراد واستدعاء validate_password(new_password, user=user) قبل تعيين كلمة المرور.

3.2 الواجهة الأمامية - 10 مشاكل عالية

Profile.tsx (109-112)

H1-FE

changePassword يرسل المعامل الخاطئ - يعيد استخدام new_password بدلاً من confirm

المعامل الثالث لـ changePassword من المفترض أن يكون new_password2 (التأكيد) لكنه يُرسل new_password مرة أخرى. قيمة حقل التأكيد الفعلي الذي أدخله المستخدم لا تُرسل أبدًا.

خطة الحل:

تغيير المعامل الثالث إلى passwordForm.confirm_password.

authStore.ts (134), Login.tsx (85)

H2-FE

عدم تطابق توقيع loginSocial - معامل ثالث غير معلن

واجهة AuthStore تعلن عن loginSocial بمعاملين، لكن التنفيذ يقبل ثلاثة بما في ذلك refresh. TypeScript لا يلتقط سوء الاستخدام عند نقاط الاستدعاء.

خطة الحل:

تحديث الواجهة لتشمل معامل refresh: string.

authStore.ts (127), App.tsx (103-104)

H3-FE

fetchProfile يُستدعى مرتين عند تسجيل الدخول

عند تسجيل الدخول: login() يستدعي fetchProfile()، ثم isAuthenticated يصبح true، ثم useEffect في App.tsx يستدعي fetchProfile() مرة أخرى. الخانق الزمني يمنع الثانية لكن هذا هش.

خطة الحل:

إزالة fetchProfile من طريقة login والاعتماد على App.tsx فقط، أو إضافة آلية تخطي.

api.ts (22, 27, 107, 164)

H4-FE

جميع دوال API تستخدم أنواع any للمعاملات

لا يوجد أمان نوعي على حمولات API. إرسال أشكال بيانات خاطئة لن يُلتقط في وقت الترجمة. مثال: إنشاء طلب بـ payment_method: 'ccp' بينما نوع CheckoutData يسمح فقط بـ 'card' | 'cod'.

خطة الحل:

استخدام واجهات TypeScript مناسبة لأجسام الطلبات (CreateOrderData، UpdateProductData، إلخ).

authService.ts (197-201)

H5-FE

registerDjango يقبل any ولا يستخدم withCredentials

لا يوجد تحقق من نوع بيانات التسجيل. لا يوجد withCredentials: true - إذا عينت الواجهة الخلفية ملفات تعريف ارتباط أثناء التسجيل فسيتم رفضها. يستخدم axios الخام بدلاً من مثيل api المُهيأ.

خطة الحل:

استخدام معامل موسوم، إضافة withCredentials: true، واستخدام مثيل api للاتساق.

OrderDetail.tsx (291-301), MerchantProductForm.tsx (337-341)

H6-FE

لا يوجد تحقق من حجم أو نوع الملف عند رفع الصور

صور إثبات الدفع وصور متغيرات المنتج تُقرأ كـ base64 عبر FileReader بدون أي تحقق من الحجم أو النوع. يمكن رفع ملف 50 ميجابايت (حوالي 67 ميجابايت base64).

خطة الحل:

إضافة فحص file.size < 5 * 1024 * 1024 و file.type.startsWith('image/') قبل القراءة.

App.tsx, main.tsx

H7-FE

لا توجد حدود خطأ React - أي خطأ يتعطل التطبيق بالكامل

لا توجد حدود خطأ React تلتف على المسارات أو المكونات. أي خطأ في أي مكون (مثل الوصول إلى undefined.seller_id) سيتعطل التطبيق بالكامل بشاشة بيضاء.

خطة الحل:

إضافة حد خطأ على الأقل في App.tsx وحدود خطأ لكل مسار للصفحات الحرجة.

emailService.ts (8)

H8-FE

emailService يستدعي خادم بريد من جانب العميل مباشرة

هذا يستدعي خادم Express منفصل على المنفذ 3002 باستخدام خدعة تحويل وكيل Vite. لن يعمل XTransformPort في الإنتاج. إذا لم يكن خادم البريد يعمل، تفشل رسائل البريد صامتًا.

خطة الحل:

نقل إرسال البريد إلى الواجهة الخلفية Django (الذي يدعم بالفعل SMTP). إزالة استدعاءات البريد من جانب العميل.

Login.tsx (216-220)

H9-FE

SuspensionModal يُعرض في صفحة تسجيل الدخول لكن لا يُفتح أبدًا

suspensionData.isOpen مهيأ لـ false. تدفق التجميد في onSubmit ينتقل مباشرة إلى /account-suspended بدلاً من فتح النافذة المنبثقة. المكون هو كود ميت يهدر موارد DOM.

خطة الحل:

استخدام النافذة المنبثقة لتدفق التجميد أو إزالة المكون والحالة بالكامل.

authService.ts (13-14)

H10-FE

رأس ngrok-skip-browser-warning مضمّن - مخاطرة أمنية في الإنتاج

رأس ngrok محدد مضمن في بناء الإنتاج. بينما غير ضار وظيفيًا، يكشف للمهاجمين أن التطبيق طُوّر باستخدام ngrok (أداة نفق)، مما قد يكشف عن تفاصيل البنية التحتية الداخلية.

خطة الحل:

استخدام متغير بيئة لتضمين هذا الرأس في التطوير فقط.

4. المشاكل المتوسطة (Medium) - ملخص

4.1 مشاكل الواجهة الخلفية المتوسطة (16 مشكلة)

الرقم	المشكلة	الملف	التأثير	خطة الحل
M1-BE	نقطة إعادة تعيين كلمة المرور تكشف وجود المستخدم	accounts/views.py (171)	تعداد المستخدمين	إرجاع نفس الرسالة دائمًا بغض النظر عن وجود المستخدم
M2-BE	لا تقييد معدل على social_login و password_reset_confirm و public_submit_appeal	accounts/views.py	قوة غاشمة، رسائل مزعجة	إضافة @throttle_classes([AuthRateThrottle]) للنقاط الثلاث
M3-BE	OTP مخزّن كنص عادي في قاعدة البيانات	accounts/models.py (183)	اختراق OTP في حالة اختراق قاعدة البيانات	تشفير OTP باستخدام make_password() والتحقق بـ check_password()
M4-BE	OTP ليس له آلية انتهاء صلاحية	accounts/models.py (179-195)	رموز OTP قديمة صالحة للأبد	إضافة حقل expires_at (10 دقائق) والتحقق منه
M5-BE	رموز JWT الموجودة تظل صالحة بعد تجميد المستخدم	accounts/services.py	تجاوز التجميد عبر رموز قديمة	إضافة رموز التحديث إلى القائمة السوداء عند التجميد
M6-BE	رابط localhost ثابت في بريد إعادة تعيين كلمة المرور	accounts/views.py (156)	رابط غير صالح في الإنتاج	قراءة URL الواجهة الأمامية من الإعدادات أو البيئة
M7-BE	التصحيح التلقائي لـ is_active يمكن إعادة تنشيط حسابات مجمّدة	accounts/serializers.py (145-150)	إعادة تنشيط غير مقصودة	إزالة التصحيح التلقائي وإصلاح السبب الجذري
M8-BE	Brand.save() لديه خطأ تعارض slug	catalog/models.py (66-69)	خطأ 500 عند إنشاء علامات تجارية	إضافة حلقة فريد مشابه لـ Category.save()
M9-BE	إعادة حساب التقييم ليست ذرية - حالة سباق	reviews/views.py	قيم تقييم غير دقيقة قليلاً	استخدام aggregate(Avg, Count) في استعلام واحد
M10-BE	cart_detail يستخدم استعلام غير محدد مسبقًا - مشكلة N+1	cart/views.py (30)	استجابة بطيئة	استخدام cart_with_prefetch() في cart_detail

الرقم	المشكلة	الملف	التأثير	خطة الحل
			تتناسب مع عدد العناصر	
M11-BE	CategorySerializer.get_children تكراري N+1	catalog/serializers.py (16)	استجابة بطيئة لشجرة الأصناف	تعليق products_count في مجموعة الاستعلام
M12-BE	Product.image تنفذ استعلامات متعددة لكل منتج	catalog/models.py (142-155)	تباطؤ شديد في صفحة قائمة المنتجات	تحديد المتغيرات والصور مسبقًا في مجموعة الاستعلام
M13-BE	لا يوجد تحقق من نوع/حجم ملف إثبات الدفع	orders/views.py (213-224)	إساءة استخدام التخزين، XSS محتمل	التحقق من الحجم (5 ميجابايت) و MIME type
M14-BE	لا يوجد ترقيم صفحات على قوائم البلاغات والاستئنافات	accounts/views.py (655, 777)	نفاد الذاكرة مع نمو البيانات	إضافة ترقيم صفحات DRF
M15-BE	UserSerializer يكشف حقل is_staff لجميع المستخدمين	accounts/serializers.py (32)	كشف معلومات عن صلاحيات المستخدم	استخدام مُسلسلات منفصلة للمشرف والمستخدم العادي
M16-BE	قيود فريدة مفقودة على BuyerReview و SellerReview	reviews/models.py (29-52)	تقييمات مكررة محتملة	إضافة unique_together = [['order', 'seller']] و[['order', 'buyer']]

4.2 مشاكل الواجهة الأمامية المتوسطة (15 مشكلة)

الرقم	المشكلة	الملف	خطة الحل
M1-FE	خانق fetchProfile يسبب بيانات قديمة لمدة 10 ثوانٍ	authStore.ts (170-190)	إعادة تعيين lastFetchProfileTime بعد تحديث الملف الشخصي
M2-FE	حساب الشحن مكرر في Cart.tsx و Checkout.tsx	Cart.tsx (135), Checkout.tsx (69)	استخراج إلى دالة مشتركة calculateShipping()
M3-FE	Home.tsx يجلب 200 منتج فقط لحساب الأصناف	Home.tsx (152)	استخدام products_count من نوع Category بدلاً من ذلك
M4-FE	OtpModal يستدعي خطافات بعد عودة مشروطة	OtpModal.tsx (16)	نقل الخطافات فوق العودة المبكرة
M5-FE	OnboardingGuard يعرض null أثناء تحميل الملف الشخصي	OnboardingGuard.tsx (20)	عرض هيكل تحميل بدلاً من null
M6-FE	ToastContainer مكرر في Layout.tsx و App.tsx	Layout.tsx (4), App.tsx (62)	إزالة ToastContainer من Layout.tsx
M7-FE	ForgotPassword.tsx يستورد sendPasswordResetEmail غير مستخدم	ForgotPassword.tsx (5)	إزالة الاستيراد غير المستخدم
M8-FE	ResetPassword.tsx خطأ في منطق تعطيل الزر	ResetPassword.tsx (131)	إضافة أقواس: disabled={loading || (!uid)}
M9-FE	Checkout.tsx لديه استدعاء navigate في جسم العرض	Checkout.tsx (72-75)	نقل إلى useEffect أو إزالته (PrivateRoute يتعامل معه)
M10-FE	Navbar.tsx MobileNavLink يستخدم نوع any	Navbar.tsx (41)	تعريف واجهة مناسبة لـ props
M11-FE	ThemeToggle يتعامل مباشرة مع DOM	themeStore.ts (16-21)	نقل معالجة DOM إلى useEffect يشاهد isDark
M12-FE	Wishlist.tsx غير محمي - يمكن الوصول بدون تسجيل دخول	App.tsx (151)	لفتها بـ PrivateRoute و OnboardingGuard
M13-FE	OrderDetail.tsx تسرب ذاكرة - URL.createObjectURL غير ملغى	OrderDetail.tsx (299)	استدعاء URL.revokeObjectURL() عند الإزالة أو إلغاء التحميل
M14-FE	ProductDetail.tsx مرجع null محتمل مع !slug	ProductDetail.tsx (576)	إضافة فحص: if (!slug) return;
M15-FE	AdminUserDetail.tsx يستخدم أنواع any بشكل مكثف	AdminUserDetail.tsx (12-16)	تعريف واجهات TypeScript مناسبة

5. المشاكل المنخفضة (Low) - 27 مشكلة

تتضمن المشاكل المنخفضة: استخدام print() بدلاً من logging في 5 مواضع (accounts/views.py)، عدم وجود ترقيم صفحات على سجل تسجيل الدخول، استيرادات متكررة لـ send_mail، إعدادات التطوير تعطل التقييد بالكامل، use_x_forwarded_host = True في الإعدادات الأساسية، رأس SameSite = None في ملفات تعريف الارتباط، تعارض معرفات الاستئناف والطلبات، تكرار عمق CategorySerializer، إنشاء tracking_number لكل طلب حتى COD، عدم وجود فهارس مركبة على LoginHistory، PaymentProof.seller يسمح بـ null، تكرار مكون ScrollToTop، حقول مفقودة في أنواع User و Profile، عدم تطابق أسماء حقول مخطط كلمة المرور، قائمة منسدلة Navbar لا تغلق عند النقر خارجها، NaN محتمل في رقم صفحة Products، خاصيات aria-label مفقودة، QueryClient مهيأ لكن نادرًا ما يُستخدم، عدم وجود تقييد معدل في ForgotPassword، Suspense يلف كل المسارات كسولًا، مكونات Skeleton معرّفة لكن غير مستخدمة، ngrok hardcoded في vite.config.ts، عنوان في AddressFields مع تبعيات غير صحيحة.

6. المخاوف المعمارية (5 قضايا)

A1. نمط أذونات غير متسق - فحوصات is_staff اليدوية في كل مكان

جميع نقاط نهاية المشرف تتحقق يدويًا من if not request.user.is_staff: return 403. هذا النمط عرضة للخطأ (نسيان إضافة الفحص = تصعيد صلاحيات). إذا نسى مطور فحصًا واحدًا، يصبحEndpoint متاحًا للمستخدمين العاديين.

خطة الحل:

إنشاء فئة إذن مخصصة IsAdminUser تتحقق من is_staff وتطبيقها عبر مزيين بشكل متسق. يمكن استخدام DRF permission_classes decorators.

A2. لا توجد طبقة تحكم صلاحيات قائمة على الأدوار

النظام يحدد User.Role (CUSTOMER, SELLER, ADMIN) في النموذج لكن لا يستخدمه أبدًا للتفويض. بدلاً من ذلك، يتم فحص is_staff و profile.is_seller بشكل عشوائي في العروض. هذا يعني أن نظام الأدوار موجود فقط في الاسم وليس في التطبيق.

خطة الحل:

تطبيق فئات إذن DRF لكل دور (IsSeller, IsBuyer, IsAdmin) وتطبيقها بشكل متسق على جميع نقاط النهاية ذات الصلة.

A3. منطق الأعمال في العروض بدلاً من الخدمات

إنشاء الطلبات (إدارة المخزون، إرسال البريد)، إنشاء التقييمات (إعادة حساب التقييم)، ومنطق إثبات الدفع كلها في العروض. فقط إجراءات المشرف تستخدم نمط AdminService. هذا يجعل العروض ضخمة وصعبة الاختبار.

خطة الحل:

استخراج منطق الأعمال إلى فئات خدمات للطلبات والتقييمات وعملية السلة. هذا يحسن قابلية الاختبار ويقلل تعقيد العروض.

A4. لا يوجد خطة آمنة لهجرات قاعدة البيانات المتزامنة

إضافة حقول جديدة بدون قيم افتراضية أو تغيير قيود ForeignKey قد تسبب مشاكل أثناء عمليات النشر بدون توقف. لا توجد استراتيجية واضحة للهجرات التدريجية.

خطة الحل:

استخدام هجرات قابلة للعكس، إضافة قيم افتراضية للحقول الجديدة، وتخطيط ترتيب الهجرات بعناية.

A5. إرسال البريد الإلكتروني متزامن - يبطئ الطلبات

جميع استدعاءات send_mail() متزامنة داخل معالجات الطلبات. إذا كان خادم SMTP بطيئًا أو غير قابل للوصول، يزداد زمن الاستجابة ويمكن أن ينتهي المهلة. يشمل ذلك رسائل الترحيب، تأكيدات الطلبات، وإشعارات الأمان.

خطة الحل:

استخدام Celery أو Django background tasks لإرسال البريد الإلكتروني بشكل غير متزامن. هذا يحرر مؤشر الترابط ويحسن زمن الاستجابة.

7. خارطة طريق إصلاح الأولويات

يُنصح بإصلاح المشاكل بالترتيب التالي بناءً على شدتها وتأثيرها. يجب إنهاء كل مرحلة بالكامل قبل البدء في المرحلة التالية.

المرحلة الأولى: إصلاح حرج فوري (1-3 أيام)

1 **أمان تسجيل الدخول الاجتماعي (C1-BE)**

إضافة التحقق من جانب الخادم لرموز Google Facebook OAuthg قبل قبول أي مطالبة هوية. هذا يمنع استحواذ الحسابات وهو أخطر ثغرة أمنية في المشروع.

2 **حالات سباق المخزون والسلة (C2-BE, C3-BE, C5-BE)**

إضافة select_for_update() على ProductVariant في عمليات إنشاء الطلب وإضافة السلة. استعادة المخزون عند إلغاء الطلب. هذه الإصلاحات تمنع خسائر مالية مباشرة.

3 **تجاوز تجديد رمز التجميد (C4-BE)**

إضافة فحص حالة المستخدم في CustomTokenRefreshView. إذا كان مجمّدًا، إضافة الرمز إلى القائمة السوداء وإرجاع 403. هذا يجعل نظام التجميد فعالًا فعلاً.

4 **إصلاحات الواجهة الأمامية الحرجة (C1-FE إلى C7-FE)**

إصلاح حفظ isAuthenticated، إزالة الرموز من sessionStorage، إصلاح تدفق تجديد الرمز، إزالة المسار المكرر، إصلاح حساب الشحن، وإضافة حقول البائع لنوع OrderItem.

المرحلة الثانية: إصلاحات عالية الأولوية (3-5 أيام)

5 **تقييد OTP وإخفاء الأخطاء (H1-BE, H3-BE)**

إضافة تقييد معدل على verify_ip_login (5 محاولات/دقيقة). استبدال str(e) برسائل خطأ عامة في جميع نقاط النهاية. منع تسريب المعلومات الداخلية.

6 **أمان الملفات وحالة الطلب (H6-BE, H4-BE)**

إضافة تحقق من نوع وحجم الملف المرفوع. تنفيذ آلة حالة صالحة للانتقالات بين حالات الطلب. منع التلاعب بالحالات.

7 **إصلاحات الواجهة الأمامية العالية (H1-FE إلى H10-FE)**

إصلاح معاملات changePassword، إضافة أمان الأنواع لدوال API، إضافة Error Boundaries، إزالة التبعية على خادم البريد من جانب العميل، إضافة تحقق من حجم الملف.

المرحلة الثالثة: تحسينات متوسطة ومنخفضة (1-2 أسبوع)

8 **الأداء وقاعدة البيانات**

إصلاح مشاكل N+1 في cart_detail g CategorySerializer g Product.image. إضافة prefetch_related g select_related حيثما ينطبق. إضافة فهارس مركبة على LoginHistory. إضافة ترقيم الصفحات على قوائم المشرف.

9 **إصلاحات متنوعة متوسطة**

تشفير OTP وإضافة expires_at. منع تعداد المستخدمين في إعادة تعيين كلمة المرور. إضافة مدققات كلمة المرور في إعادة التعيين. استبدال print() بـ logging. إصلاح رابط localhost في بريد إعادة التعيين.

المرحلة الرابعة: تحسينات معمارية (2-3 أسابيع)

10

إعادة هيكلة معمارية

إنشاء طبقة أذونات قائمة على الأدوار (IsAdmin, IsSeller, IsBuyer). استخراج منطق الأعمال إلى فئات خدمات (OrderService, ReviewService). نقل إرسال البريد إلى مهام خلفية عبر Celery. توحيد نمط التحقق من الأذونات في جميع نقاط النهاية.